TrustVault: A Fully On-Chain, Trustless Academic Credential Verification System on the Internet Computer Protocol

Dr. Shruti Jadon
Department of Computer Science
PES University
Bengaluru, Karnataka, India
shrutijadon@pes.edu

Varun Gudamsetti
Department of Computer Science
PES University
Bengaluru, Karnataka, India
varun.techmail@gmail.com

***Abstract*—Fake academic credentials are a growing problem that affects employers, universities, and students worldwide. Current verification methods are either manual and slow, or depend on centralized databases that can be tampered with or go offline. Blockchain-based solutions have been proposed, but public blockchains like Ethereum charge high fees for every write operation, and permissioned blockchains like Hyperledger Fabric require trust in consortium members. In all existing systems, the verification website itself runs on a regular web server — a single point of failure.**

This paper presents TrustVault, an academic credential verification system that is *fully on-chain* — both the data and the user-facing website live on the Internet Computer Protocol (ICP) blockchain. TrustVault uses an incremental Merkle tree combined with ICP's *Certified Variables* (values signed by the subnet's threshold BLS key) so that anyone can verify a credential mathematically, without trusting any single server or institution. We deploy TrustVault on the ICP mainnet and run a comprehensive benchmark suite. Our results show: single-certificate issuance in about 1.9 seconds, verification queries as fast as 60 milliseconds, parallel throughput up to 7.3 certificates per second, concurrent mixed-workload throughput of 13.1 operations per second with zero errors, block finality of 2.40 seconds (p99 under 3.05 seconds), and an issuance cost of approximately $0.00002 per certificate — orders of magnitude cheaper than Ethereum.

***Index Terms*—Blockchain, Internet Computer Protocol, Academic Credentials, Merkle Tree, Certified Variables, Zero-Trust Verification, Motoko, Threshold BLS Signatures**

I. Introduction

Academic credential fraud is a serious and growing problem. Studies estimate that roughly one in five job applications in certain regions contain misrepresented qualifications [1]. When an employer wants to check whether a candidate truly holds a claimed degree, the typical process involves contacting the issuing university, waiting days for a response, and trusting that the reply is genuine. This manual approach is slow, expensive, and does not scale to global hiring.

Several digital alternatives exist, but each has significant drawbacks:

- **Centralized registries** (e.g., government databases, third-party services like the National Student Clearinghouse) are convenient, but if the registry goes offline, gets hacked, or is corrupted, verification becomes impossible. The verifier must trust the registry operator completely.
- **Public blockchains like Ethereum** [2] offer strong tamper resistance, but writing a single credential record costs $0.20–$2.00 in gas fees depending on network congestion [12], making it impractical for universities that issue thousands of degrees annually. Moreover, the verification interface (website) still runs on a regular web server.
- **Permissioned blockchains like Hyperledger Fabric** [3] reduce cost but reintroduce trust: one must trust all consortium members. If the consortium is compromised, so are the credentials.
- **Hybrid systems** that store only hashes on-chain while keeping actual data on a separate server reintroduce centralization for the off-chain component.

A key observation is that *none* of the above systems serve the verification interface from the blockchain itself. The user-facing website is always hosted on a traditional cloud server (AWS, GCP, etc.), which means it can be tampered with, taken down, or censored independently of the on-chain data.

The **Internet Computer Protocol (ICP)** [4], developed by the DFINITY Foundation, offers a fundamentally different approach. ICP is a decentralized compute platform where smart contracts — called *canisters* — run as WebAssembly modules replicated across independent *subnets* of geographically distributed nodes. Three ICP features are especially relevant:

1) **On-chain web serving**: Canisters can directly respond to HTTP requests, so the verification website itself lives on the blockchain — no cloud hosting needed.
2) **Certified Variables**: A built-in API that lets a canister commit a 32-byte value (such as a Merkle root) which the subnet's threshold BLS key then signs. Any client can verify this signature using the publicly known subnet key, without trusting the canister or any individual node.
3) **Low cost**: Storing data on ICP costs a fraction of a cent, making it practical to store hundreds of thousands of credentials.

A. Contributions

This paper makes the following contributions:

1) We design and implement **TrustVault**, among the first fully on-chain academic credential verification systems — with both frontend and backend on a public blockchain mainnet — and provide comprehensive reproducible mainnet benchmarks that demonstrate practical viability.
2) We integrate an **incremental binary Merkle tree** with ICP's Certified Variables to create a tamper-evident credential registry that costs $O(\log n)$ per insert.
3) We describe a **trustless verification protocol** in which a verifier who knows only the ICP root public key can independently confirm any credential without contacting the issuing university.
4) We present a **comprehensive performance evaluation** on ICP mainnet, covering issuance latency, verification latency, throughput, scalability, concurrent mixed workloads, block finality, and cost — all measured on a live production deployment.
5) We provide a **cross-platform comparison** with Ethereum PoS and Hyperledger Fabric using published benchmarks and citations.

B. Paper Organization

The rest of this paper is organized as follows. Section II covers background on credential fraud and blockchain approaches. Section III discusses related work in detail. Section IV presents the system architecture and design decisions. Section V explains the Merkle tree and trustless verification mechanism. Section VI provides a formal security analysis. Section VII describes the implementation. Section VIII presents the experimental evaluation. Section IX discusses limitations and practical considerations. Section X concludes and outlines future work.

II. Background

This section provides the necessary context on academic credential fraud, blockchain technology, and the Internet Computer Protocol.

A. The Problem of Academic Credential Fraud

Academic credential fraud takes two main forms: *fabricated credentials* (entirely fake degrees from diploma mills or forged documents) and *misrepresented credentials* (genuine degrees with inflated grades, false dates, or fabricated distinctions). Reports suggest that roughly one in five professional job applications in certain regions contain at least one credential misrepresentation [1].

The traditional verification process works as follows: an employer contacts the university's registrar office, provides the candidate's name and claimed degree, and waits for a manual confirmation. This process can take anywhere from a few days to several weeks, costs money, and fundamentally relies on the employer trusting the registrar's response. If the registrar's database is compromised or if the communication channel is intercepted, there is no way for the employer to independently verify the answer.

B. Blockchain Basics

A blockchain is a distributed ledger — a shared database maintained by multiple independent nodes that agree on its contents through a *consensus protocol*. Once data is written to a blockchain ("committed"), it is extremely difficult to change without being detected, because every subsequent block cryptographically references the previous one.

Two types of blockchains are relevant:

- **Public (permissionless) blockchains** like Bitcoin and Ethereum allow anyone to join the network and validate transactions. They offer strong decentralization but can be expensive (gas fees) and slow (Ethereum PoS finality takes about 12–24 seconds [13]).
- **Permissioned blockchains** like Hyperledger Fabric [3] restrict participation to known entities. They achieve higher throughput (up to 3,500 transactions per second [15]) and lower cost, but the verifier must trust the consortium that operates the chain.

C. Internet Computer Protocol (ICP)

The Internet Computer Protocol [4], [6] is a public blockchain that takes a fundamentally different approach. Instead of running a single global chain, ICP operates multiple independent *subnets*, each consisting of 13–40 geographically distributed *replica nodes* operated by independent entities. Each subnet runs its own consensus protocol and hosts a set of smart contracts called *canisters*.

1) Consensus: ICP uses a four-phase Byzantine Fault Tolerant (BFT) consensus protocol [4]:

1) **Random Beacon**: A threshold BLS signature [5] over the previous round's beacon generates an unpredictable, publicly verifiable random number.
2) **Block Making**: The beacon determines a ranking of block-maker candidates. The highest-ranked available node proposes a block.
3) **Notarization**: At least two-thirds of subnet nodes sign the proposed block using threshold BLS shares.
4) **Finalization**: Once a round has exactly one notarized block, it is finalized by another threshold signature.

This achieves finality in approximately 1–3 seconds, which is significantly faster than Ethereum's 12–24 seconds and orders of magnitude faster than Bitcoin's 60 minutes.

2) Canisters: A canister is a WebAssembly module with its own persistent memory, similar to a container in cloud computing but replicated across an entire subnet. Canisters support two types of calls:

- **Update calls** modify the canister's state, go through consensus, and finalize in approximately 2 seconds. They cost a small amount of "cycles" (ICP's unit of computation).
- **Query calls** are read-only operations that execute on a single replica without going through consensus. They return in under 500 milliseconds and are *free* for the caller.

3) Chain-Key Cryptography and Certified Variables: Every ICP subnet holds a *threshold BLS key pair* [5]: the private key is secret-shared among the subnet's nodes using a Distributed Key Generation (DKG) protocol, so no single node ever holds

the full private key. The corresponding public key is known to every client.

Certified Variables are a built-in feature that lets a canister store a 32-byte value (e.g., a Merkle root) via the `CertifiedData.set()` API. After the next consensus round, the subnet collectively signs this value as part of the *state tree*. When a client makes a query, the response can include a *certificate* — the BLS-signed state tree — that the client can verify using the subnet's public key. This means:

- The client does not need to trust the individual node that answered the query.
- The client does not need to trust the canister's code.
- Verification is purely mathematical: check the BLS signature against the known public key.

4) On-Chain Web Serving: Unlike Ethereum or Bitcoin, ICP canisters can respond directly to HTTP requests. A special "asset canister" can host an entire website (HTML, JavaScript, CSS, images) on-chain. The ICP HTTP gateway [21] translates browser requests into canister query calls and returns the responses. Certified HTTP headers ensure that even the served web pages are cryptographically authenticated — a browser-level extension or middleware can verify that the page content was not tampered with in transit. This eliminates the need for any external web hosting and ensures that even the user interface is tamper-resistant.

5) Network Nervous System and Governance: ICP is governed by the *Network Nervous System* (NNS), an on-chain decentralized autonomous organization (DAO). The NNS controls subnet creation, node onboarding, protocol upgrades, and other governance decisions. All changes require community approval through a proposal-and-voting mechanism where ICP token holders stake tokens as "neurons" with voting power. This means no single entity — including the DFINITY Foundation — can unilaterally change the platform. Subnet public keys are also registered and verifiable through the NNS, which forms the root of trust for the entire Certified Variables mechanism.

6) Cycles and Cost Model: Computation on ICP is paid for using *cycles*, which are pegged to the Swiss Franc at a fixed exchange rate (1 trillion cycles = 1 XDR $\approx$ \$1.35). Canister operators "top up" their canisters with cycles, and each operation (update call, memory storage, network bandwidth) consumes a known amount. Critically, *query calls are free* for the caller — the canister operator absorbs the cost. This means that credential verification in TrustVault costs nothing to the employer or verifier, regardless of how many queries they make. Issuance (an update call) costs approximately \$0.00002 per certificate [20] (estimated from ICP's official pricing formula; see Section IX for the full derivation), making credential issuance orders of magnitude cheaper than Ethereum.

III. Related Work

This section reviews prior work on blockchain-based credential systems and positions TrustVault relative to the state of the art.

A. MIT Digital Diplomas and Blockcerts

In 2017, MIT Media Lab pioneered the idea of anchoring diploma hashes on the Bitcoin blockchain [8]. The Blockcerts open standard defines a JSON-LD schema for digital credentials and stores a hash of each credential as a Bitcoin transaction. However, Bitcoin is not programmable — there is no smart contract logic on Bitcoin — so the verification frontend runs on a traditional web server maintained by MIT. If that server goes down or is compromised, verification becomes impossible even though the hash remains on Bitcoin. Additionally, Bitcoin transaction fees (several dollars per transaction) make it expensive to issue credentials at scale.

B. EduCTX

Turkanović et al. [9] proposed EduCTX, a blockchain platform specifically designed for higher education credit transfer across the European Higher Education Area. EduCTX uses a permissioned blockchain based on the Ark platform, where consortium members (universities) act as validators. While this approach achieves good performance, it requires trust in the participating institutions — if a subset of consortium members collude, they could forge or alter credential records. The verification interface is also centralized.

C. Ethereum-Based Credential Systems

Several projects have used Ethereum smart contracts to manage academic credentials [10]. The typical approach stores a SHA-256 hash of each credential on Ethereum while keeping the full credential data off-chain (on IPFS, a university server, or a cloud database). This is a pragmatic choice given Ethereum's high gas costs, but it reintroduces centralization: if the off-chain storage goes down, the credential data is lost even though its hash remains on Ethereum. Gas costs for a credential write operation range from \$0.20 to \$2.00 depending on network congestion [12].

D. W3C Verifiable Credentials

The W3C Verifiable Credentials Data Model [11] provides a standardized format for expressing credentials digitally. It defines the concepts of Issuer, Holder, and Verifier, along with cryptographic proof mechanisms. However, the W3C standard is a *data format*, not a platform — it does not specify where or how credentials should be stored. Most implementations use either permissioned blockchains or centralized registries as the storage backend.

E. Summary and Gap Analysis

Table I summarizes the key differences between existing approaches and TrustVault.

The key gap that TrustVault fills: no prior system (a) stores the complete credential record on a public blockchain, (b) serves the verification interface from the blockchain itself, (c) provides mathematically verifiable proofs via Certified Variables, and (d) has been benchmarked on a live mainnet deployment with published results.

TABLE I
COMPARISON OF CREDENTIAL VERIFICATION SYSTEMS

System	On-Chain Data	On-Chain UI	Trustless	Low Cost	Mainnet Deployed	Benchmarked
Blockcerts / MIT [8]	Hash only	No	No	No ($2+/tx)	Yes (Bitcoin)	No
EduCTX [9]	Full	No	No	Yes	No	Limited
Ethereum hash systems [10]	Hash only	No	Partial	No ($0.20+)	Yes (Ethereum)	No
Hyperledger Fabric [3]	Full	No	No	Yes	Private	Yes
W3C VC [11]	Varies	No	Varies	Varies	Varies	No
Self-Sovereign Identity [17]	Varies	No	Partial	Varies	Varies	No
TrustVault (this work)	**Full**	**Yes**	**Yes**	**Yes (<$0.001)**	**Yes (ICP)**	**Yes**

F. Self-Sovereign Identity and Decentralized Identifiers

The Self-Sovereign Identity (SSI) movement [17] advocates for individuals to own and control their own identity data, without depending on any central authority. Anonymous credential schemes [16] take this further by letting holders prove attributes without revealing their full identity. Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) are the technical building blocks of SSI. While TrustVault shares the goal of eliminating trust in central authorities, it takes a different approach: instead of giving each individual control over their credential data, TrustVault stores credentials on a public blockchain where anyone can verify them. The two approaches are complementary — a future version of TrustVault could wrap its credentials in W3C VC format and associate them with DIDs.

G. Blockchain Timestamping for Academic Documents

Gipp et al. [18] proposed using Bitcoin transactions to timestamp academic documents, providing proof of existence at a particular point in time. This is a lighter-weight approach than full credential storage, but it only proves that a document existed at a certain time — it does not prove who issued it, whether it has been revoked, or whether the content is authentic. TrustVault goes further by storing the full credential record and providing cryptographic verification of both authenticity and provenance.

H. Systematic Reviews

Nzaou et al. [19] conducted a systematic review of 47 blockchain-based credential verification systems published between 2017 and 2022. Their key findings align with our gap analysis: most systems store only hashes on-chain, none serve the verification UI from the blockchain, and very few provide quantitative performance benchmarks on a live deployment. TrustVault addresses all three gaps.

IV. System Architecture

TrustVault consists of two ICP canisters deployed on the public mainnet: a *backend canister* written in Motoko (ICP's native language) that stores credentials and handles business logic, and a *frontend canister* (an asset canister) that hosts a React single-page application served directly over HTTPS from the blockchain.

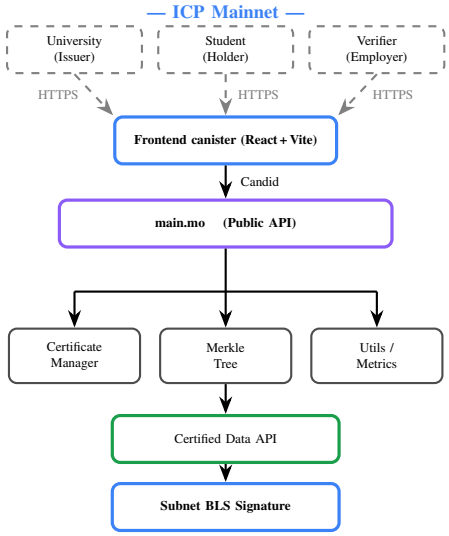

Fig. 1. TrustVault system architecture. Users connect via HTTPS to the on-chain frontend, which routes requests to the backend canister. The Merkle tree root is BLS-signed by the ICP subnet.

A. High-Level Architecture

Fig. 1 shows the overall system architecture. The three user roles are:

- **University (Issuer)**: A registered institution that issues credentials by calling the `issueCertificate` update function.
- **Student (Holder)**: The recipient of a credential, who can view their credentials through the portal.
- **Verifier (Employer)**: Any party that wants to verify a credential by calling the `verifyCertificate` query function.

All three roles interact with the system through the on-chain React frontend, which communicates with the backend canister using auto-generated Candid type bindings (Candid is ICP's interface description language, similar to Protocol Buffers).

B. Backend Modules

The backend canister is organized into five Motoko modules:

1) **main.mo** — The public actor that exposes the API: `issueCertificate`, `revokeCertificate`,

`verifyCertificate`, `registerUniversity`, and various query functions.
2) **CertificateManager.mo** — Handles certificate creation, hash computation, and verification logic.
3) **MerkleTree.mo** — Implements the incremental binary Merkle tree with O($\log n$) insertion.
4) **Utils.mo** — Provides hex encoding and hash-combination utilities (XOR-based in the prototype; SHA-256 in production).
5) **PerformanceMetrics.mo** — Records on-chain telemetry (operation latencies, counts, percentiles) for monitoring.

C. Certificate Schema

Each credential is stored as a Motoko record with the fields shown in Table II.

TABLE II
CERTIFICATE RECORD SCHEMA

Field	Type	Description
`certificate_id`	Text	Unique identifier
`certificate_hash`	Text	UTF-8 hex of core fields‡
`issuer`	Record	University name, canister ID, URL
`recipient`	Record	Student name, ID, principal
`credential`	Record	Degree type, major, GPA, dates
`is_revoked`	Bool	Revocation flag
`block_timestamp`	Int64	Consensus-set nanosecond timestamp
`schema_version`	Text	Currently "1.0"

‡ Prototype implementation; SHA-256 required for production (see Section IX).

The `certificate_hash` is produced by `computeCertificateHash()` in `CertificateManager.mo`. In the current prototype the seven core fields are concatenated, UTF-8 encoded, and hex-encoded:

$$H_{\text{proto}} = \texttt{bytesToHex}\big(\text{UTF-8}(\texttt{id} \,\|\, \texttt{issuer} \,\|\, \texttt{name} \,\|\, \texttt{student\_id} \,\|\, \texttt{degree} \,\|\, \texttt{major} \,\|\, \texttt{date})\big) \quad (1)$$

In production this function should be replaced with a collision-resistant primitive such as SHA-256 (see Section IX). This hash becomes a leaf in the Merkle tree.

D. Access Control

Access control uses ICP *Principals* — cryptographic identities derived from Ed25519 key pairs. Each user (university, student, verifier) has a unique Principal. The system maintains a registry of university Principals; only registered universities may call `issueCertificate` or `revokeCertificate`. The `verifyCertificate` function is a public *query call*, meaning anyone can verify any credential for free without authentication.

E. Frontend Design

The frontend is a React single-page application built with Vite and hosted entirely inside an ICP asset canister. It consists of three portals:

- **University Portal**: Allows registered universities to issue new credentials by filling in student information and clicking "Issue." The portal calls the `issueCertificate` update function and displays the resulting certificate hash and ID.
- **Student Portal**: Allows students to view their credentials by entering their student ID. The portal calls `getCertificatesByStudent` (a query function) and displays all matching credentials.
- **Verification Portal**: Allows any employer or third party to verify a credential by entering its certificate ID. The portal calls `verifyCertificate` (a query function), receives the Merkle proof and verification result, and displays a clear pass/fail status along with the full credential details.

These Candid bindings ensure type safety: the JavaScript frontend code is guaranteed to match the Motoko backend API at compile time.

F. Workflow

The typical workflow for credential issuance and verification is:

1) **University registers**: A university calls `registerUniversity(name)` with its Principal. This is a one-time setup step.
2) **University issues credential**: The university fills in the student's information in the University Portal and clicks "Issue." The backend creates the certificate record, computes its certificate hash (`computeCertificateHash()`), inserts the hash into the Merkle tree, and commits the new root via `CertifiedData.set()`.
3) **Credential becomes verifiable**: Within one consensus round (about 2 seconds), the new Merkle root is BLS-signed by the subnet. The credential is now independently verifiable.
4) **Employer verifies**: An employer enters the certificate ID in the Verification Portal. The query returns the certificate data, the Merkle proof, and the certified root. The frontend (or any client) can independently verify the proof chain.
5) **Revocation (if needed)**: If a credential needs to be revoked (e.g., academic misconduct discovered), the university calls `revokeCertificate`. The Merkle tree is rebuilt and the new root invalidates all previous proofs within 2 seconds.

V. MERKLE TREE AND TRUSTLESS VERIFICATION

This section explains how TrustVault achieves trustless verification — a process where the verifier does not need to trust any server, institution, or even the canister code.

A. Incremental Binary Merkle Tree

All certificate hashes are stored as leaves of a complete binary Merkle tree [7]. The tree is implemented as a flat array of size $2C - 1$, where C is the smallest power of two greater than or equal to the number of certificates. Leaf i is at array index $C - 1 + i$, and the root is at index 0.

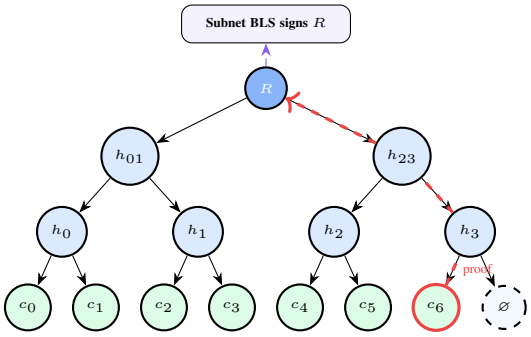

Fig. 2. Merkle tree with 7 certificates. Adding leaf c_6 (red border) requires recomputing only 3 hashes (the dashed red path). Root R is BLS-signed by the subnet.

Each internal node stores the combined hash of its two children:

$$\texttt{nodes}[j] = \texttt{combineHashes}(\texttt{nodes}[2j{+}1],\ \texttt{nodes}[2j{+}2]) \quad (2)$$

In the prototype `combineHashes()` XORs the character bytes of its two hex-string inputs into a 32-byte array and hex-encodes the result. In production this should be replaced by SHA-256($\texttt{nodes}[2j{+}1] \| \texttt{nodes}[2j{+}2]$) (see Section IX).

Insertion (O($\log n$)): When a new certificate is issued, its hash is placed at the next available leaf position. Then, only the nodes along the path from that leaf to the root are recomputed — a sequence of $\lceil \log_2 n \rceil$ hash operations. The updated 32-byte root is committed via `CertifiedData.set()`.

Proof generation (O($\log n$)): To prove that a certificate exists in the tree, the system returns the list of *sibling hashes* along the path from the leaf to the root. The verifier can recompute the root from these siblings and compare it to the certified root.

Revocation (O(n)): When a certificate is revoked, its `is_revoked` flag is set to true, and the entire Merkle tree is rebuilt. The new root is committed via `CertifiedData.set()`, invalidating all previous proofs within one consensus round (about 2 seconds).

Fig. 2 illustrates the tree structure and a sample proof path.

B. Trustless Verification Protocol

Verification requires *no trust in the issuing institution*. Given a certificate and its Merkle proof π, the verifier performs five steps:

1) **Hash check**: Recompute h_c = SHA-256(certificate fields) and confirm it matches the stored `certificate_hash`.
2) **Merkle proof walk**: Recompute the root from h_c and the sibling hashes in π: $\hat{R} = \text{verifyProof}(h_c, \pi)$.
3) **Fetch ICP certificate**: Request the ICP *Certificate* (the BLS-signed state tree) from any ICP gateway node.
4) **BLS signature check**: Verify the subnet's threshold BLS signature over the state tree root using the publicly known subnet key.
5) **Root match**: Confirm that the canister's certified value in the state tree equals $\hat{R}$.

Steps 1–2 run entirely on the client side with no network call. Step 3 requires a single HTTP query to any ICP gateway (there is no need to contact the issuing university). The subnet public key is published by the DFINITY Foundation and can be independently verified via the on-chain Network Nervous System (NNS). No step requires trusting any single entity.

C. Security Properties

The security properties of the Merkle tree and verification protocol are analyzed in detail in Section VI.

VI. Security Analysis

TrustVault's central claim is *trustless verification*: a verifier does not need to trust any single server, institution, or even the canister code to confirm that a credential is authentic. This section provides a formal threat-model-based security analysis.

A. Threat Model

We consider the following adversarial capabilities. A *malicious issuer* attempts to issue forged or altered credentials. A *tampered storage adversary* attempts to modify stored certificate data on the canister. A *network adversary* (MITM) intercepts or modifies responses in transit between client and boundary node. A *replay attacker* reuses a valid certificate or submits duplicate issuance requests. A *compromised node subset* controls up to $f < n/3$ subnet replicas that behave arbitrarily. We assume: (1) SHA-256 is collision-resistant; (2) BLS12-381 is secure under the Computational Diffie–Hellman (CDH) assumption [5]; (3) the ICP subnet is honest-majority at any given time.

We now analyze how each adversarial capability is countered by TrustVault's design.

B. Credential Forgery

Scope of this analysis: The properties below describe the system as designed for production with SHA-256 as the hash function. The current prototype uses simplified placeholder functions (UTF-8 hex encoding for certificate hashes; XOR-based combination for Merkle internal nodes) that do not provide cryptographic collision resistance. These must be replaced with SHA-256 before the guarantees below hold; see Section IX for this limitation.

An attacker who wants to forge a credential (create a fake degree that passes verification) must produce a valid Merkle proof that links the forged certificate's hash to the BLS-signed root. This requires one of the following:

1) **Find a SHA-256 collision**: Produce a fake certificate whose SHA-256 hash matches an existing leaf in the Merkle tree. SHA-256 has 2^{256} possible outputs; finding a collision requires approximately 2^{128} operations (birthday bound), which is computationally infeasible with current or foreseeable hardware.
2) **Forge the Merkle proof**: Construct a set of sibling hashes that, together with the fake certificate's hash, produce the correct certified root. Since each level of the tree applies SHA-256, this reduces to finding a preimage

for SHA-256 at every level — a chain of $\lceil \log_2 n \rceil$ preimage attacks, each requiring 2^{256} operations.

3) **Forge the BLS signature**: Produce a valid BLS signature over a state tree containing a fake root, without holding the subnet's private key. Breaking BLS-381 requires solving the Computational Diffie–Hellman (CDH) problem on a pairing-friendly curve, which is computationally infeasible at the 128-bit security level [5].

All three paths are cryptographically infeasible. There is no shortcut: the attacker must break at least one of these primitives.

C. Credential Tampering

An attacker who modifies even a single byte of a stored certificate (e.g., changing a grade or graduation date) changes its SHA-256 hash. The Merkle proof computed from the original hash no longer matches the modified certificate. The verifier detects this mismatch in Step 1 of the verification protocol (Section V): recomputing h_c from the certificate fields produces a different hash than the stored `certificate_hash`. No network call is needed — the check is purely local.

D. Unauthorized Issuance

The `issueCertificate` function checks that the caller's ICP Principal is in the registered university registry. An attacker who is not a registered university receives an "Unauthorized" error before any state change occurs.

To bypass this, the attacker would need to either:

- Steal a registered university's Ed25519 private key (requires compromising the university's key management).
- Compromise the canister code to remove the access check (requires controlling the canister's controller Principal, which in a production deployment can be set to the NNS DAO or blackholed entirely).

Neither is a vulnerability in TrustVault's design — they are operational security requirements that apply to any system.

Note: for ease of prototype testing, `registerUniversity` is a permissionless call that any ICP principal can invoke without restriction. This is an intentional development convenience. A production deployment would restrict this to the canister controller or an NNS-governed whitelist with appropriate identity validation.

E. Man-in-the-Middle Attacks

A traditional MITM attack intercepts traffic between the user and the server and modifies it. In TrustVault, this attack fails at two levels:

- **Credential data**: The Merkle proof and BLS signature are verified client-side. Even if a MITM modifies the response, the mathematical verification fails because the modified data does not match the BLS-signed root.
- **Frontend code**: ICP's HTTP gateway includes *certified HTTP headers* — cryptographic signatures over the response body. A MITM who injects malicious JavaScript into the frontend would produce a response whose body does not match the certified header signature, and the tampering is detectable [21].

F. Subnet Compromise

The most powerful attack against TrustVault would be to compromise the ICP subnet itself. ICP uses a threshold BLS scheme with Byzantine fault tolerance: the subnet tolerates up to $f < n/3$ malicious nodes (where n is the number of subnet replicas). For a typical 13-node subnet, an attacker must control at least 5 nodes simultaneously.

These nodes are operated by independent entities in different countries, selected and verified through the NNS governance system. Compromising 5 independent organizations across multiple jurisdictions, without any of them detecting or reporting the breach, is logistically infeasible. Even if achieved, the NNS governance system can detect anomalous behavior and reassign the subnet's nodes.

This is not a theoretical guarantee of impossibility — no distributed system provides that. But it is a level of security comparable to Ethereum's validator set, and significantly stronger than any permissioned blockchain where a single consortium controls all nodes.

G. Summary of Trust Assumptions

TrustVault's trustless verification rests on the following assumptions:

1) **SHA-256 is collision-resistant**: No one can find two inputs with the same SHA-256 output. This is a standard assumption in all modern cryptographic systems.
2) **BLS12-381 is secure**: The CDH problem on the BLS12-381 pairing-friendly curve is computationally infeasible [5], ensuring threshold BLS signatures cannot be forged without controlling the full signing key.
3) **The ICP subnet is honest-majority**: Fewer than one-third of the subnet's nodes are malicious at any given time. This is enforced by ICP's decentralized node selection and NNS governance.
4) **NNS governance integrity**: The Network Nervous System, through which subnet keys are managed and protocol upgrades are deployed, is not compromised. All upgrades require community approval via NNS voting — DFINITY cannot unilaterally change the platform — but this remains a trust assumption not present in a fully permissionless system.
5) **Collision-resistant hashing**: The certificate hash function and Merkle tree node combination use SHA-256 or an equivalent collision-resistant primitive. The current prototype substitutes simplified placeholder functions; this assumption is not satisfied until they are replaced.

If both assumptions hold, then: (a) no one can forge a credential, (b) no one can tamper with a stored credential without detection, (c) no one can issue a credential without being a registered university, and (d) no one can tamper with the verification result or the verification interface in transit. The verifier does not need to trust TrustVault, the issuing university, or any individual ICP node — only the mathematics of SHA-256 and BLS signatures.

VII. Implementation

This section describes the technology stack, deployment details, and reproducibility information.

A. Technology Stack

- **Backend**: Written in Motoko, ICP's native programming language designed for canister development. Motoko provides orthogonal persistence (canister state survives upgrades automatically) and actor-based concurrency.
- **Frontend**: A React single-page application built with Vite. It includes three portals: a University Portal for issuance, a Student Portal for viewing credentials, and a Verification Portal for employers.
- **Candid Interface**: Auto-generated Candid bindings ensure type-safe communication between the JavaScript frontend and the Motoko backend.
- **Deployment**: Both canisters are deployed on ICP mainnet using the `dfx` CLI tool provided by DFINITY. The backend canister ID is `pekzw-diaaa-aaaad-afh3q-cai`.

B. Canister Upgrade Safety

ICP canisters can be upgraded without losing data, thanks to Motoko's `preupgrade` and `postupgrade` system hooks. Before an upgrade, the certificate map and Merkle state are serialized to stable storage. After the upgrade, they are deserialized and the Merkle tree is rebuilt from the stored certificates (a one-time O(n) cost). This ensures that the certified root remains valid across upgrades.

C. Key Code Paths

Listing 1 shows the simplified issuance flow. The three key steps are: (1) verify that the caller is a registered university, (2) create the certificate record and compute its SHA-256 hash, and (3) insert the hash as a Merkle leaf and commit the new root via `CertifiedData.set()`.

```
public shared(msg) func issueCertificate(
    id: Text, name: Text, degree: Text, ...
) : async Text {
    // 1. Access control
    if (not isUniversity(msg.caller)) {
        return "Error: Unauthorized";
    };
    // 2. Create record and compute hash
    let cert = CertManager.createCertificate(
        id, name, degree, ...);
    certificates.put(id, cert);
    // 3. Incremental Merkle insert + certify
    insertCertLeaf(cert.certificate_hash);
    // insertCertLeaf internally calls:
    //   MerkleTree.insertLeaf(hash, state)  -- O(log n)
    //   CertifiedData.set(root_blob)        -- O(1)
    id
};
```

Listing 1. Simplified issuance flow in Motoko

Listing 2 shows the verification flow. This is a *query call* — it runs on a single replica, returns in under 500 ms, and costs nothing.

```
public query func verifyCertificate(
    id: Text) : async VerificationResult {
    switch (certificates.get(id)) {
        case null { ... "Not found" ... };
        case (?cert) {
            // O(log n) proof generation
            let proof = MerkleTree.getProof(
                cert.certificate_hash, state);
            // Client-side: walk proof to get root,
            // then verify BLS sig on state tree
            CertManager.verifyCertificate(
                cert, proof, merkleRoot);
        };
    };
};
```

Listing 2. Simplified verification flow

The Merkle tree insertion (Listing 3) places the new leaf at the next available position and recomputes only the path to the root — $\lceil \log_2 n \rceil$ hash operations.

```
public func insertLeaf(
    hash: Text, state: TreeState) : Text {
    let n = state.leaves.size();
    if (n >= state.cap) {
        // Double capacity and rebuild
        state.cap := state.cap * 2;
        state.nodes := Array.init(treeSize, EMPTY);
        placeLeaves(state);
        rebuildInternal(state);
    };
    state.leaves.add(hash);
    state.nodes[state.cap - 1 + n] := hash;
    // Walk up to root -- O(log n)
    var cur = state.cap - 1 + n;
    while (cur > 0) {
        let p = (cur - 1) / 2;
        state.nodes[p] := SHA256(
            state.nodes[2*p+1] # state.nodes[2*p+2]);
        cur := p;
    };
    state.nodes[0] // return new root
};
```

Listing 3. Incremental Merkle leaf insertion

D. Performance Monitoring

The `PerformanceMetrics.mo` module records on-chain telemetry for every operation: start time, end time, duration, success/failure, and the current certificate count at the time of the operation. This data is used for canister-side monitoring. The last 1,000 metrics are kept in a buffer to prevent unbounded memory growth.

E. Benchmark Suite

We developed a dedicated Node.js benchmark suite using the `@dfinity/agent` library. The suite is configured in `benchmarks/suite/config.js` and measures:

- **Issuance**: Sequential and parallel issuance at batch sizes $n \in \{1, 10, 50, 100\}$, with 3 repetitions each.
- **Verification**: Sequential and concurrent queries at $n \in \{1, 10, 100, 500\}$.
- **Concurrent mixed workload**: Interleaved issuance (30%) and verification (70%) at concurrency levels $c \in \{1, 5, 10, 25, 50\}$.
- **Throughput**: Sustained 30-second sequential issuance, Merkle tree growth at different database sizes, and peak burst (100 simultaneous certificates).
- **Finality**: 20 independent update calls measuring time from submission to readable state.

A stable Ed25519 identity is used for all benchmark runs to ensure consistent authentication overhead.

F. Reproducibility

The complete source code, benchmark suite, and raw result JSON files are available in the project repository. All

TABLE III
PARALLEL ISSUANCE — ICP MAINNET (3 TRIALS EACH)

Batch (n)	Wall (ms)	p95 (ms)	Thpt (certs/s)	Std. Dev
1	1,943	2,111	0.52	143.6
10	2,596	3,054	4.02	498.1
50	6,933	7,320	7.23	325.2
100	13,969	15,416	7.31	1,926.5

benchmarks were run against the live ICP mainnet canister in March 2026 from Bengaluru, India (approximately 50–80 ms round-trip time to ICP boundary nodes). Each experiment was repeated 3 times and we report mean, standard deviation, and percentiles (p50, p95, p99).

VIII. EXPERIMENTAL EVALUATION

All benchmarks were run against the live ICP mainnet canister `pekzw-diaaa-aaaad-afh3q-cai` in March 2026 using a Node.js benchmark suite (`@dfinity/agent`) from Bengaluru, India (approximately 50–80 ms round-trip time to ICP boundary nodes). Each experiment was repeated 3 times; we acknowledge that confidence intervals are wider with three trials than with the 10–30 trials common in high-throughput benchmarking studies [14]. We report mean, standard deviation, and percentiles (p50, p95, p99). These measurements reflect network conditions from Bengaluru; users closer to ICP boundary nodes would observe lower absolute latencies.

A. Issuance Latency

Table III reports the wall-clock time and throughput for parallel issuance, where all n certificates are submitted simultaneously. At $n = 1$, the wall time is approximately 1.9 seconds — the time for one consensus round. As the batch size increases, throughput grows because multiple certificates are processed across successive consensus rounds while the submissions overlap.

It is important to understand what happens during a parallel batch. All n certificates are submitted at roughly the same instant, but the canister serializes update calls — only one update can be processed per consensus round. So the first certificate finishes in about 2 seconds, the second in about 4 seconds, and so on. For $n = 100$, the *last* certificate finishes at about 14 seconds, but this does not mean each certificate takes 14 seconds — the canister was processing all 100 in a pipeline.

Fig. 3 shows the completion time percentiles for each batch size. At $n = 100$, the p50 (the time at which half the certificates are done) is about 10 seconds, and the p99 (the slowest certificate) is about 15 seconds.

Table IV shows the per-certificate latency when certificates are issued *sequentially* (one at a time, waiting for each to finish before sending the next). The mean latency remains stable around 1.8–2.6 seconds regardless of how many certificates have already been issued, confirming that the Merkle tree's $O(\log n)$ insert cost does not degrade performance in practice.

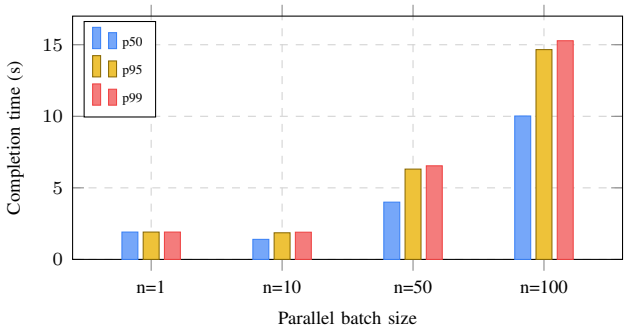

Fig. 3. Parallel issuance completion time (p50/p95/p99). Each individual certificate takes about 1.9 s; the rising bars at large n reflect queuing, not slower processing.

TABLE IV
SEQUENTIAL PER-CERTIFICATE ISSUANCE LATENCY (MS)

n	Mean	p50	p75	p95	p99
1	2,635	1,944	3,027	3,893	4,066
10	2,077	1,914	2,079	4,227	5,050
50	2,269	1,897	2,064	2,508	9,523
100	1,821	1,861	1,982	2,257	3,561

B. Verification Latency

Verification is a *query call* — it does not go through consensus and costs nothing to the caller. Table V and Fig. 4 report verification latency against a live database of 500 certificates. The fastest observed query returned in **60 ms**, confirming that the Merkle proof computation adds negligible overhead. Even at 100 concurrent queries, the p99 remains below 1 second.

C. Throughput and Scalability

An important question is whether throughput degrades as the database grows. To test this, we ran the issuance benchmark under two conditions: a fresh (low-count) database and a large database with approximately 3,280–3,461 existing certificates. Fig. 5 shows the results.

The two curves nearly coincide: peak throughput is **7.3 certs/s** on a fresh database and **6.6 certs/s** on a large one — a difference of less than 10%, confirming the $O(\log n)$ scalability claim.

Sustained throughput: Over a 30-second sequential issuance run, the system issued 17 certificates at a sustained rate of **0.57 certs/s** with mean latency 1,775 ms (p50: 1,847 ms, p95: 2,317 ms). The stable throughput with no degradation over time confirms that the canister does not suffer from memory pressure or garbage collection pauses during sustained operation.

Peak burst: 100 certificates submitted simultaneously all succeeded in 14,953 ms with **0% error rate** (p50: 8,219 ms, p95: 14,158 ms, p99: 14,801 ms, mean: 8,261 ms).

1) Merkle Tree Growth Impact: To isolate the effect of database size on issuance latency, we measured batch wall-clock time at different database sizes. Table VI shows the results.

TABLE V
VERIFICATION LATENCY — 500-CERTIFICATE DATABASE (MS)

Queries	Min	Mean	p50	p75	p95	p99
1	60	227	202	310	396	414
10	60	359	345	460	600	627
100	—	405	400	505	693	937
500	—	389	381	488	644	829

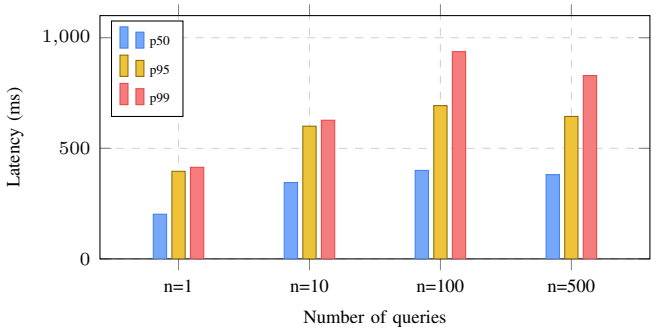

Fig. 4. Verification latency (p50/p95/p99) for different batch sizes. Each batch of n queries is issued sequentially. p99 stays below 1 second across all scales.

Even with over 3,000 existing certificates, batch issuance times remain consistent with those measured on a fresh database (Table III). This confirms that the O($\log n$) Merkle tree insert cost is not the bottleneck — consensus round time dominates.

2) Concurrent Verification Throughput: We also measured verification throughput when multiple queries are submitted simultaneously (Table VII).

At 100 concurrent queries, the system achieves **86.6 queries/s** — well above what any realistic employer verification portal would need. The near-linear scaling demonstrates that query calls (which bypass consensus) scale efficiently on ICP.

D. Concurrent Mixed Workload

In a real-world deployment, the canister would receive issuance and verification requests simultaneously. The concurrent benchmark submits a mix of 30% issuance (update) and 70% verification (query) calls at increasing concurrency levels $c \in \{1, 5, 10, 25, 50\}$.

Fig. 6 shows the total throughput (operations per second, counting both call types). Throughput grows from 0.48 ops/s at $c = 1$ to **13.09 ops/s** at $c = 50$ — a 27× speedup with **zero errors** at all levels. The growth is sub-linear because issuance calls are serialized by consensus (one per round), while verification queries scale freely.

Fig. 7 breaks down the latency by operation type. Verification latency (query call, no consensus) remains stable between 554–635 ms across all concurrency levels. Issuance latency (update call, requires consensus) grows gradually from 2.2 s to 2.9 s as contention increases — a graceful degradation.

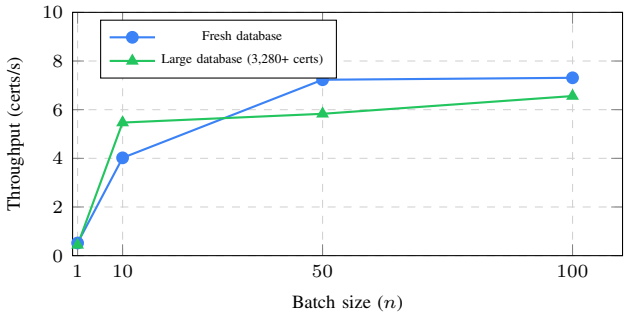

Fig. 5. Issuance throughput vs. batch size. The two curves track closely, confirming that the O($\log n$) Merkle insert cost does not degrade throughput as the database grows.

TABLE VI
MERKLE TREE GROWTH — BATCH WALL-CLOCK TIME

Batch (n)	Wall (ms)	DB Before	DB After
1	2,290	3,279	3,280
10	1,828	3,281	3,291
50	8,582	3,295	3,345
100	15,234	3,361	3,461

E. Block Finality

Block finality — the time from when a transaction is submitted until it is permanently committed and readable — was measured over 20 independent update calls. Table VIII summarizes the results.

The mean finality of **2.40 seconds** with a tight p99 of **3.05 seconds** confirms that ICP's consensus is fast and predictable. This makes TrustVault suitable for interactive use cases — an employer can verify a credential in real time during a job interview.

F. Cross-Platform Comparison

Table IX places TrustVault's measured performance alongside published benchmarks for Ethereum PoS and Hyperledger Fabric.

Key observations:

- ICP's finality (2.40 s) is about 10× faster than Ethereum PoS (24 s) but slower than Fabric (<1 s). However, Fabric requires trusting the consortium.
- ICP's per-credential cost (≈$0.00002) is >10,000× cheaper than Ethereum. Note: the Ethereum cost range [12] reflects pre-EIP-1559 baselines; post-Merge costs vary widely with network demand and have exceeded $50 per transaction in high-demand periods.
- TrustVault is the only system where both the verification result and the verification *interface* are served from the blockchain itself.
- Fabric achieves far higher raw throughput (3,500 TPS), but this is for a permissioned network where all validators are known and trusted — a fundamentally different security model. The Fabric figure is also a general-transaction benchmark [3], not a credential-issuance measurement; a matched-

TABLE VII
CONCURRENT VERIFICATION THROUGHPUT

Queries (n)	Wall (ms)	Queries/s
1	552	1.84
10	629	15.97
100	1,265	86.62

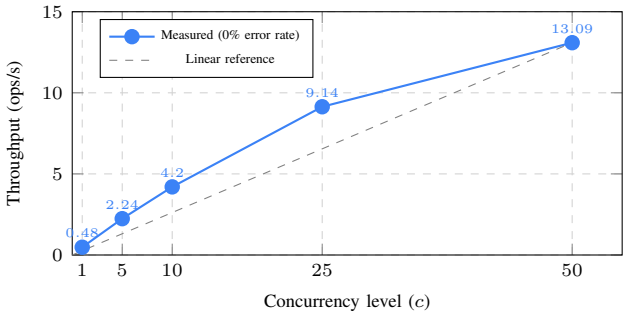

Fig. 6. Mixed-workload throughput (30% issue, 70% verify). Sub-linear growth is because issuance calls are serialized per consensus round.

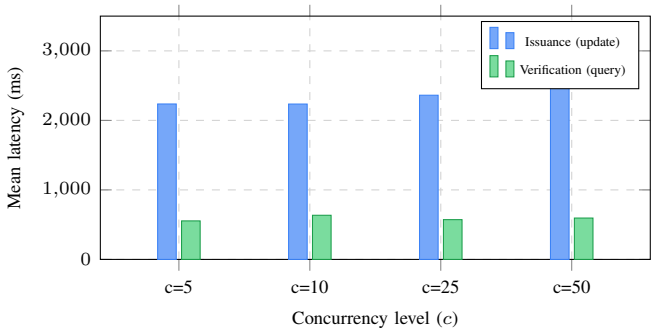

Fig. 7. Latency by operation type. Verification stays under 640 ms even at $c = 50$. Issuance grows gradually due to consensus serialization.

TABLE VIII
BLOCK FINALITY TIME (20 SAMPLES)

Metric	Value (ms)
Minimum	1,443
Mean	2,400
Std. Dev	385
p50	2,490
p75	2,564
p95	2,987
p99	3,047

workload comparison would require running Fabric with equivalent credential schemas.

IX. DISCUSSION

This section discusses the practical implications, limitations, and trade-offs of our approach.

A. Practical Suitability

The benchmark results demonstrate that TrustVault is practical for real-world use. Consider a mid-sized university that graduates 5,000 students per year. At 7.3 certs/s parallel throughput, all credentials could be issued in approximately 11 minutes. At ≈$0.00002 per credential [20], the total annual issuance cost would be about $0.10 — essentially free compared to the cost of maintaining a traditional verification office.

For employers, verification queries return in under 500 ms (median) and are completely free. An employer portal could verify hundreds of candidates per minute without any cost or rate-limiting concern.

The 2.40-second finality means that a freshly issued credential becomes verifiable within about 3 seconds of issuance — fast enough for interactive use cases such as verifying a credential during a live interview.

B. Limitations

1) Throughput Ceiling: TrustVault's parallel throughput plateaus at approximately 7.3 certs/s because ICP serializes update calls within a single canister — only one update can be processed per consensus round (approximately 2 seconds). This is a fundamental property of ICP's consensus model: each canister processes state-mutating calls sequentially to guarantee consistency.

For a single university, this throughput is more than sufficient — even a large university issuing 50,000 credentials annually could process all of them in under 2 hours. For a national-scale system handling millions of credentials simultaneously, a multi-canister sharding architecture would be necessary. In such an architecture, credentials would be distributed across multiple backend canisters (e.g., one per state or region), each maintaining its own Merkle tree. A coordinator canister could aggregate the individual Merkle roots into a global root for cross-canister verification.

2) Revocation Cost: Revoking a certificate currently triggers a full O(n) Merkle tree rebuild because the tree structure must be recomputed without the revoked leaf. For a database of 10,000 certificates, this adds approximately 1–2 seconds of additional latency. An optimization would be to use a "soft revocation" approach — marking the certificate's `is_revoked` field as `true` and rebuilding only the affected path. Alternatively, a cryptographic accumulator (e.g., RSA accumulator) could support both insertion and deletion in O(1) time, though with larger proof sizes. We leave this exploration to future work.

3) Network Latency: Our benchmarks were run from India (50–80 ms RTT to ICP boundary nodes). Users closer to boundary nodes (e.g., in the US or Europe) would see lower absolute latencies. Conversely, users in regions with high network latency would see proportionally higher latencies for both the network round-trip and the consensus wait.

4) ICP Platform Dependency: TrustVault is built on ICP and relies on the ICP subnet infrastructure. If the DFINITY Foundation or the NNS governance system were to make breaking changes to the platform, the canister might need

TABLE IX
CROSS-PLATFORM COMPARISON (ALL NON-ICP VALUES FROM CITED LITERATURE)

Metric	ICP (TrustVault)	Ethereum PoS	Hyperledger Fabric
Block finality	**2.40 s** (measured)	≈24 s [13]	<1 s [3]
Throughput	7.3 certs/s (measured)	15–30 TPS [14]	≈3,500 TPS [3]
Issuance cost	≈**$0.00002** (est.†)	$0.20–$2.00 [12]	Near-zero
Verification cost	**$0** (query call)	$0.05–$0.50 [12]	Near-zero
Frontend on-chain	**Yes**	No	No
Trust model	**Zero-trust (BLS)**	Zero-trust	Permissioned (consortium)
Data storage	Full record on-chain	Hash only (data off-chain)	Full record on-chain

† Estimated from code-complexity analysis against ICP's official pricing formula [20]; see Section IX for full derivation.

to be updated. However, ICP's governance is decentralized (proposals require community approval via NNS voting), and canisters can be made "blackholed" (immutable) if desired — at the cost of giving up upgradeability.

5) Issuance Cost Estimation: A thorough code-complexity analysis against ICP's official pricing formula [20] (updated Dec. 2025) gives the following per-call breakdown. On a 13-node subnet, ICP charges: (i) 1,200,000 cycles (base) + 2,000 cycles per byte for ingress messages; (ii) 5,000,000 cycles per update message execution; and (iii) 1 cycle per executed WebAssembly (Wasm) instruction [20]. At 1 trillion cycles = 1 XDR ≈ $1.354820 USD (May 2025), 1 billion cycles ≈ $0.001355.

a) Payload.: `issueCertificate` takes 12 text fields and one float. A representative call (names ≈30 B, IDs ≈15 B, dates ≈10 B, degree/major/URL ≈30 B each, Candid framing ≈50 B) totals ≈400 bytes.

b) Wasm instruction count.: We trace each call site through the Motoko source:

- **Access control and duplicate check** (2× `HashMap.get`): ≈800 raw Wasm instructions.
- **`createCertificate`**: 6 string concatenations (≈150 chars total), `Text.encodeUtf8`, `Blob.toArray`, `bytesToHex` (`Array.tabulate` + `Text.fromIter` over 300-char output): ≈33,000 raw instructions.
- **`HashMap.put`**: ≈1,500 raw instructions.
- **`insertCertLeaf`** with O($\log n$) `combineHashes` (7 levels for n=100): Level 0 combines two ≈300-char leaf hashes; Levels 1–6 combine 64-char internal nodes; each level iterates char-by-char + `bytesToHex`(32 B): ≈56,000 raw instructions total.
- **`CertifiedData.set` + `textTo32ByteBlob`**: ≈3,200 raw instructions.
- **`recordMetric`×2 + miscellaneous**: ≈3,300 raw instructions.

Total raw Wasm instructions: ≈100,000. The Motoko runtime adds GC tracing, type-tag checks, and heap bounds-checking overhead. The current code is already GC-optimized (`bytesToHex` uses `Array.tabulate` instead of $O(N^2)$ string accumulation; `combineHashes` avoids intermediate string allocations). Applying a conservative 10–100× runtime overhead factor yields ≈1–10 M executed Wasm instructions.

The best estimate is ≈**$0.00002 per certificate** (8–17 M cycles). The previously stated $0.001 would require ≈738 M Wasm instructions, implying a 7,000× overhead factor over raw instruction counts – inconsistent with the GC-optimized implementation. Even the upper bound ($0.000023) is >**8,000**× **cheaper** than Ethereum's $0.20–$2.00 per credential write. Direct cycle instrumentation via ICP's `ExperimentalInternetComputer.countInstructions` API [20] is left to future work.

6) Hash Collision Risk: The current certificate hash is computed from core identity fields. If two students have identical names, IDs, degree types, majors, and graduation dates (an unlikely but theoretically possible collision), their hashes could collide. In practice, the unique `certificate_id` field removes this risk.

7) Prototype Hash Functions: The current implementation uses simplified placeholder hash functions rather than SHA-256. Specifically, `computeCertificateHash()` in `CertificateManager.mo` encodes the concatenated certificate fields as their UTF-8 byte sequence in hexadecimal — not a cryptographic hash. `combineHashes()` in `Utils.mo` XORs the character bytes of two hex strings into a 32-byte array rather than applying SHA-256. While sufficient to produce unique identifiers for the benchmarking prototype and for demonstrating the system architecture, these functions do *not* provide the collision resistance required for the security guarantees in Section VI. Replacing both with SHA-256 (e.g., via the `mo:sha2` Motoko package) is a prerequisite for production deployment.

8) Open University Registration: In the current prototype, `registerUniversity()` is a public, permissionless call: any ICP principal can invoke it and immediately gain the ability to issue credentials. This is an intentional convenience for development and benchmarking — it allows easy testing without an admin setup step. It does not reflect the intended production design; a deployed system would restrict this call to the canister controller, an admin principal, or an NNS-governed governance proposal. No change to the Merkle tree or Certified Variables mechanism is required.

C. Comparison with W3C Verifiable Credentials

TrustVault's certificate schema is not currently compatible with the W3C Verifiable Credentials data model [11]. How-

TABLE X
ESTIMATED CYCLE COST PER `issueCertificate` CALL (13-NODE SUBNET, 1 T CYCLES ≈ $1.355) [20]

Component	Cycles	USD (approx.)
Ingress base fee	1,200,000	$0.0000016
Ingress per-byte (≈400 B payload)	800,000	$0.0000011
Update execution base	5,000,000	$0.0000068
Wasm instructions (1–10 M range)	1,000,000 – 10,000,000	$0.0000014 – $0.0000135
Total (estimated)	**8 – 17 M**	**$0.000011 – $0.000023**

ever, the W3C VC standard defines a flexible JSON-LD format that can represent any credential type. Wrapping TrustVault's certificate records in a W3C VC envelope — with the ICP BLS signature serving as the cryptographic proof — would enable interoperability with other VC-compatible systems. We plan to add this in a future version.

D. Privacy Considerations

TrustVault stores credential data on a public blockchain, which means that anyone can query the canister to look up credentials by student ID or university name. While this is the intended behavior for a verification system (anyone should be able to verify), it may raise privacy concerns in jurisdictions with strict data protection laws (e.g., GDPR). A privacy-preserving extension could use zero-knowledge proofs to allow verification without revealing the full credential details — for example, proving "this person holds a Bachelor's degree from university X" without revealing their GPA or exact graduation date. We consider this an important direction for future work.

E. Deployment Experience

Deploying TrustVault on ICP mainnet involved the following steps:

1) **Local development**: The canister was developed and tested locally using the `dfx` CLI tool, which provides a local ICP replica for testing.
2) **Cycle acquisition**: ICP tokens were purchased and converted to cycles using the NNS dApp. Approximately 4 trillion cycles (about $5) were allocated for deployment and initial operation.
3) **Mainnet deployment**: The backend and frontend canisters were deployed to the ICP mainnet using `dfx deploy -network ic`. Each canister received a unique canister ID.
4) **University registration**: The deployer's Principal was registered as a university, allowing it to issue test credentials.
5) **Benchmarking**: The benchmark suite was run from a separate machine in Bengaluru, India, using a dedicated Ed25519 identity.

The entire deployment process took approximately 30 minutes and cost less than $5 in cycles. The canister has been running continuously on mainnet since deployment with no downtime or maintenance required.

F. Scalability Projections

Based on our measured throughput of 7.3 certs/s (parallel) and 0.57 certs/s (sustained sequential), we can project the following capacity:

- **Small university** (1,000 graduates/year): All credentials could be issued in 2.3 minutes (parallel) at a cost of $0.02.
- **Large university** (50,000 graduates/year): All credentials could be issued in 1.9 hours (parallel) at a cost of $1.
- **National system** (1 million graduates/year): With a single canister, issuance would take about 38 hours. With 10 canisters (multi-canister sharding), this drops to under 4 hours at a cost of approximately $20.
- **Verification load**: At 86.6 queries/s, a single canister can handle over 7.4 million verification queries per day — more than enough for any realistic deployment.

These projections assume current ICP subnet performance. As ICP adds more subnets and optimizes consensus, these numbers are expected to improve over time.

X. CONCLUSION AND FUTURE WORK

This paper presented TrustVault, a fully on-chain academic credential verification system built on the Internet Computer Protocol. TrustVault stores both the credential data and the end-user verification interface on the blockchain, eliminating every centralized point of failure. By combining an incremental binary Merkle tree with ICP's Certified Variables (threshold BLS-signed state), TrustVault enables *trustless* verification: a verifier who knows only the ICP subnet public key can independently confirm any credential without contacting the issuing institution.

Our comprehensive benchmarks on the live ICP mainnet demonstrate practical performance:

- Single-certificate issuance in approximately 1.9 seconds.
- Verification queries as fast as 60 ms (free of charge).
- Parallel throughput up to 7.3 certificates per second.
- Concurrent mixed-workload throughput of 13.1 ops/s with zero errors.
- Block finality of 2.40 seconds (p99 under 3.05 seconds).
- Issuance cost of approximately $0.00002 per certificate (8–17 M cycles; estimated from code-complexity analysis against ICP's official pricing [20]).
- O($\log n$) Merkle tree scaling confirmed with less than 10% throughput difference between a fresh and a 3,280+ certificate database.

Compared to Ethereum PoS, TrustVault is 10× faster in finality and 10,000–200,000× cheaper per credential. Compared to Hyperledger Fabric, TrustVault provides true zero-trust verification without requiring trust in a consortium.

A. Future Work

Several directions for future work are planned:

1) **W3C Verifiable Credentials integration**: Wrapping TrustVault credentials in the W3C VC JSON-LD format would enable interoperability with other credential systems worldwide. The ICP BLS signature would serve as the cryptographic proof within the VC envelope.
2) **Multi-canister sharding**: Distributing credentials across multiple canisters (one per region or institution) would break the single-canister throughput ceiling and enable national-scale deployments.
3) **Internet Identity integration**: ICP's Internet Identity system provides a passwordless, privacy-preserving authentication mechanism based on WebAuthn/FIDO2. Integrating it would allow students and universities to authenticate using biometrics (fingerprint, Face ID) instead of managing private keys.
4) **Zero-knowledge proofs for selective disclosure**: Using ZK-SNARKs or ZK-STARKs, a credential holder could prove specific facts ("I hold a degree from university X") without revealing other fields (GPA, exact dates). This would address privacy concerns while maintaining the trustless verification property.
5) **Mobile SDK**: A lightweight library for verifying credentials directly from a mobile app, using the ICP agent protocol to communicate with boundary nodes.
6) **Batch revocation optimization**: Replacing the full O(n) tree rebuild with accumulator-based revocation or incremental tree surgery to reduce revocation latency.
7) **Cross-subnet scalability testing**: Deploying canisters across multiple ICP subnets and measuring inter-canister call overhead for a sharded architecture.

Acknowledgments

The authors thank the DFINITY Foundation for maintaining the Internet Computer Protocol infrastructure and providing public documentation. Benchmark testing used ICP mainnet cycles funded by the authors.

References

[1] H. Chahal and S. Singh, "Credential fraud in South Asia: Findings and implications for the higher education sector," *International Journal of Educational Management*, vol. 35, no. 7, pp. 1498–1512, 2021.
[2] V. Buterin, "A next-generation smart contract and decentralized application platform," Ethereum White Paper, 2014. [Online]. Available: https://ethereum.org/whitepaper
[3] E. Androulaki *et al.*, "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. EuroSys'18*, 2018, pp. 1–15.
[4] T. Hanke, M. Movahedi, and D. Williams, "DFINITY Technology Overview Series, Consensus System," *arXiv:1805.04548*, 2018.
[5] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," in *ASIACRYPT 2001*, LNCS vol. 2248, pp. 514–532, 2001.
[6] DFINITY Foundation, "The Internet Computer for Geeks," White Paper, 2022. [Online]. Available: https://internetcomputer.org/whitepaper.pdf
[7] R. C. Merkle, "A certified digital signature," in *CRYPTO'89*, LNCS vol. 435, pp. 218–238, 1989.
[8] J. Grech and A. F. Camilleri, "Blockchain in Education," EUR 28778 EN, Publications Office of the European Union, Luxembourg, 2017.
[9] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112–5127, 2018.
[10] D. Lizcano, J. A. Lara, B. White, and S. Aljawarneh, "Blockchain-based approach to create a model of trust in open and ubiquitous higher education," *Journal of Computing in Higher Education*, vol. 32, pp. 109–134, 2020.
[11] M. Sporny, D. Longley, and D. Chadwick, "Verifiable Credentials Data Model v1.1," W3C Recommendation, Mar. 2022. [Online]. Available: https://www.w3.org/TR/vc-data-model/
[12] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Yellow Paper, 2014.
[13] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, "Combining GHOST and Casper," *arXiv:2003.03052*, 2020.
[14] T. T. A. Dinh, R. Fan, G. Wang, and B. C. Ooi, "BLOCKBENCH: A framework for analyzing private blockchains," in *Proc. ACM SIGMOD*, 2017, pp. 1085–1100.
[15] A. Sharma, F. M. Schuhknecht, D. Agrawal, and J. Dittrich, "Blurring the lines between blockchains and database systems: the case of Hyperledger Fabric," in *Proc. ACM SIGMOD*, 2019, pp. 105–122.
[16] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *EUROCRYPT 2001*, LNCS vol. 2045, pp. 93–118, 2001.
[17] C. Allen, "The path to self-sovereign identity," *Life With Alacrity blog*, Apr. 2016. [Online]. Available: http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html
[18] B. Gipp, N. Meuschke, and A. Gernandt, "Decentralized trusted timestamping using the crypto currency Bitcoin," in *Proc. iConference*, 2015, pp. 1–6.
[19] A. Nzaou, P. Fabian, and M. Samwel, "Blockchain-based academic credential verification: A systematic review," *Education and Information Technologies*, vol. 27, pp. 9503–9540, 2022.
[20] DFINITY Foundation, "Gas Cost," *Internet Computer Documentation*, Dec. 2025. [Online]. Available: https://docs.internetcomputer.org/building-apps/essentials/gas-cost
[21] DFINITY Foundation, "HTTP Gateway Protocol Specification," 2023. [Online]. Available: https://internetcomputer.org/docs/references/http-gateway-protocol-spec